

CS747 - Programming Assignment 2

Avinash Chaudhari - 23B1064

October 2025

1 Task 1: MDP Planning Algorithms

Problem Overview

In this task, two classical algorithms for solving finite Markov Decision Problems (MDPs) are implemented: Howard's Policy Iteration (HPI) and Linear Programming (LP). The assignment requires parsing MDP instances, computing the optimal value function and policy using the specified algorithm, and optionally evaluating a given policy file.

Design and Implementation

MDP Parsing: The program extracts the number of states and actions, terminal states, all nonzero transitions, the task type (episodic/continuing), and the discount factor γ . Parsing is robust to arbitrary paths and strictly follows the provided file format.

Howard's Policy Iteration (HPI):

- *Policy Evaluation:* Given a policy π , the linear system $V^\pi(s) = \sum_{s',r,p} p[r + \gamma V^\pi(s')]$ is solved for all non-terminal states, either exactly or to a tolerance of 10^{-8} .
- *Policy Improvement:* For each state, all actions are evaluated using the Bellman optimality equation and the policy is updated by selecting the action with maximum expected value. All improvable states are switched simultaneously as per Howard's rule.
- Repeat evaluation and improvement until the policy converges. Terminal states are handled by fixing their values and skipping in updates.

Linear Programming (LP):

- The LP method minimizes the sum of state values subject to Bellman optimality constraints for all actions:

$$V(s) \geq \sum_{s',r,p} p[r + \gamma V(s')] \quad \forall s, a$$

- Terminal states are fixed to zero (or a specified value) and removed from the constraints.
- The PuLP Python library is used as the LP solver backend.

Policy Evaluation: To evaluate a provided policy, the implementation iteratively computes V^π until convergence using the same approach as in policy iteration.

Output Format: Outputs are printed as S lines: each **value** <tab> **action**, with at least 6 decimal places, meeting autograder requirements.

Algorithmic Choices and Handling

- **Tie-breaking:** If multiple actions yield equal value during improvement, any of them may be chosen.
- **Terminal States:** Terminal states are omitted from updates and their value is fixed at 0 or a designated reward.
- **Convergence Threshold:** A small tolerance (10^{-8}) is used to determine convergence in iterative routines.

Observations

- Policy iteration converges quickly on all given MDPs, typically in fewer than 50 iterations.
- LP formulation guarantees optimality and handles large state spaces efficiently within provided constraints.
- The modular approach allows easy integration with the autograder and checks output format precisely.

2 Task 2: MDP Encoding for Strategic Card Play

State Space Representation

Let the deck consist of 26 cards: numbers 1–13, each appearing as a heart (H) and a diamond (D). The agent’s hand is represented by a vector $\mathbf{h} = (h_1, h_2, \dots, h_{13})$, where $h_i \in \{0, 1, 2\}$ denotes the number of cards of value i in the hand, regardless of suit. This abstraction is justified because both the scoring (including the special sequence bonus) and all actions depend only on card values, not suits.

The state space $\mathcal{S}$ is thus the set of all such vectors $\mathbf{h}$ such that the sum of the hand, $\sum_{i=1}^{13} i \cdot h_i$, is strictly less than the threshold T . All hands with sum $\geq T$ are collapsed into a single terminal “bust” state. This pruning reduces the

number of states from 3^{13} to a much smaller set (e.g., 2466 for $T = 26$), making the problem tractable.

Action Space Representation

The action set $\mathcal{A}$ is defined as follows:

- **Add** ($a = 0$): Draw a random card from the remaining deck and add it to the hand. If the resulting hand sum exceeds or equals T , transition to the terminal state.
- **Swap** ($a = 1$ to $a = 13$): Swap out a card of value k ($k = a$) from the hand (if present) with a random card from the remaining deck. The swap is only allowed if $h_k > 0$.
- **Stop** ($a = 14$): End the game voluntarily, transitioning to the terminal state.

Transition and Reward Structure

For each non-terminal state $\mathbf{h}$ and action a :

- **Add**: For each possible card c that can be drawn (with $2 - h_c$ copies remaining), the transition probability is proportional to the number of such cards left in the deck. If adding c causes the hand sum to reach or exceed T , the next state is the terminal state with reward 0 (bust). Otherwise, the next state is $\mathbf{h}'$ with $h'_c = h_c + 1$ and reward 0.
- **Swap**: If $h_k = 0$, the action is invalid. Otherwise, for each possible card c to be drawn, the transition probability is as above. The next state is $\mathbf{h}'$ with $h'_k = h_k - 1$ and $h'_c = h_c + 1$. If the new hand sum $\geq T$, transition to the terminal state with reward 0; otherwise, reward 0.
- **Stop**: Transition to the terminal state with reward equal to the hand sum plus the bonus (if the hand contains the specified consecutive sequence), or just the hand sum otherwise.

There was a choice of giving the reward at every transition instead of giving the accumulated reward at the end. I tried that but that did not give the same answers so implemented this.

Justification for the Encoding

This encoding is efficient and sufficient for the following reasons:

- **Suit Irrelevance**: Since both the bonus and all actions depend only on card values, not suits, tracking only the count of each value is sufficient.
- **State Pruning**: By collapsing all bust states into a single terminal state, the state space is dramatically reduced, enabling tractable computation.

- **Action Validity:** Only meaningful actions are allowed in each state (e.g., swaps are only possible if the card is present in the hand).
- **Reward Structure:** The reward function accurately models the game rules, including the special sequence bonus and bust condition.

This formulation allows standard MDP solvers (Howard’s Policy Iteration, Linear Programming) to efficiently compute optimal policies for the card game.

References

- Source code and algorithm descriptions from lecture notes, code documentation, and Sutton & Barto’s *Reinforcement Learning: An Introduction*.